Project Title: Intelligent Fraud Detection and Financial Decision System

Author(s): Vedashree Bane, Brady Duncan

Northeastern University

DS 5500 Capstone: Application in Data Science

Spring 2026

Date: April 20, 2026

Abstract

A brief summary of the project, including the problem statement, methodology, key findings, and conclusions. Keep it concise, around 200-300 words.

0. Contents

1. Introduction

1.1 Purpose of Methodology

Fraud detection presents a critical challenge because existing models frequently miss fraudulent transactions. In this context, recall is the most important evaluation metric. Every undetected fraud case carries severe financial and operational consequences. Our methodology therefore prioritizes maximizing recall on validation data, ensuring as few fraudulent transactions as possible slip through undetected.

1.2 Problem Statement

With online purchases and data leaks rapidly increasing, spenders need a reliable system to quickly and effectively detect fraud. Existing frameworks are often hard to navigate or confusing, so this dashboard aims to clarify such processes. Additionally, the dashboard will give the user spending insights and guides, such as credit and spending tracking.
We chose this problem because we are both interested in the industry of finance or consulting, where there is a massive amount of data with countless problems to address. We felt that such a problem would allow us to implement industry-relevant solutions based on advanced machine learning and data visualization concepts.

1.3 Related Work

Our work is built upon existing research in fraud detection, ensemble learning, explainable artificial intelligence (XAI), and financial decision systems.

Khalid et al. [1] proposed an ensemble voting classifier combining Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Random Forest, Bagging, and Boosting techniques to address class imbalance in credit card fraud detection. Their approach achieved an accuracy of 99.96% on SMOTE-balanced datasets. However, the study did not explore feature selection strategies, nor an-

alyze real-time scalability, which are critical for deployment in production-grade financial systems.

Compagnino et al. [2] presented a comprehensive review of over 120 fraud detection studies, analyzing traditional machine learning, ensemble, and deep learning approaches. Their findings revealed that Random Forest models achieved high precision (93%) but very low recall (34%) on real-world banking datasets, highlighting the difficulty of identifying rare fraud patterns using supervised methods alone. The authors emphasized the importance of evaluation metrics such as Area Under the Precision-Recall Curve (AUPRC), precision, recall, and F1-score over accuracy when dealing with highly imbalanced datasets.

An Integrated Multistage Ensemble Machine Learning (IMEML) model was introduced in the Journal of Big Data [3], combining multiple ensemble architectures with data balancing techniques such as Random Under Sampling and Cluster Centroids. While the framework demonstrated improved fraud detection capability, the study did not report detailed performance metrics and lacked explainability mechanisms, limiting its applicability in regulated financial environments.

Research on credit scoring and loan default prediction using behavioral and transactional data [4] compared Logistic Regression, Random Forest, XGBoost, LightGBM, and Neural Networks, integrating SHAP and LIME for model interpretability. The study found that behavior-derived scores, geolocation patterns, and merchant-level features were strong predictors of default risk. However, it did not address challenges related to data privacy, missing values, or real-world deployment constraints.

Finally, recent work on Natural Language Processing (NLP) for financial risk detection [5] proposed a text preprocessing and feature extraction pipeline using Bag-of-Words and TF-IDF representations combined with machine learning classifiers. The study demonstrated that sentiment signals from financial text could serve as early indicators of fraud risk, but lacked concrete performance benchmarks and did not address adversarial manipulation or multilingual robustness.

Collectively, these studies demonstrate that effective fraud detection systems require integrated approaches combining ensemble learning, feature engineering, data balancing, and explainability. Persistent challenges remain in achieving adequate recall, handling concept drift, and scaling to real-world production environments, gaps that our project addresses through comprehensive ensemble modeling, credit scoring integration, SHAP-based explainability, and interactive dashboards for financial decision-making.

1.4 Dataset

The dataset we selected for our project was the **IEEE-CIS Fraud Detection Dataset** from Kaggle. It consists of a collection of credit card transactions along with merchant, credit card, and other information for each transaction.

We selected this information because it is the leading dataset for fraud de-

tectors, which we have researched in our related works. It will allow us to consider many different transaction characteristics when training our model, as well as spending habits. Additionally, we plan to calculate a risk/trust score for our dashboard, which will, for each credit card, mathematically aggregate all transactions along with their fraud risk and other factors to give an estimate about how risky the transactions on that card have been overall. The dataset is also very large, with 871 features across all files, allowing us to consider many factors in both our fraud detection and risk calculation. Overall, we will be able to use this dataset to build more than what past fraud-detection datasets have achieved and create a reliable, applicable product.

The dataset consists of two files: **Transaction** and **Identity**. They are linked by TransactionID, which is unique for each transaction. The IEEE-CIS Fraud Detection dataset contains 434 features in the transaction data and 41 features in the identity data, totaling 871 features across all files. Transaction features include essential information such as transaction amount (TransactionAmt), product code (ProductCD), card details (card1-card6), transaction addresses (addr1, addr2), distance measurements (dist1, dist2), and email domain information (P_emaildomain, R_emaildomain). Additionally, the dataset includes numerous engineered features categorized as C1-C14 (counts), D1-D15 (time deltas), M1-M9 (match indicators), and V1-V339 (Vesta-engineered features capturing transaction patterns and device characteristics). Identity features provide supplementary information about digital identifiers, device types (DeviceType, DeviceInfo), network connection details (id_12-id_38), and user identification data. This feature set enables comprehensive analysis of transaction patterns, user behavior, device characteristics, and temporal sequences all necessary for accurate fraud detection and credit risk assessment.

The dataset can be found at [6].

2. Methodology and Model Development

2.1 Preprocessing

The preprocessing pipeline transforms the raw IEEE-CIS Fraud Detection dataset into model-ready feature matrices. Each transformation was fit exclusively on the training set and applied afterward to the validation set to prevent data leakage.

2.1.1 Data Loading and Merging

The pipeline begins by merging the Transaction and Identity files on their shared `TransactionID` key using a left join. This preserves all transaction records while attaching identity information where available, since not every transaction has a corresponding identity entry.

2.1.2 Temporal Train-Validation Split

To prevent data leakage and simulate a realistic deployment scenario, the dataset was split chronologically rather than randomly. Transactions were sorted by `TransactionDT` (with `TransactionID` as a tiebreaker), and the earliest 80% of transactions were assigned to the training set while the most recent 20% formed the validation set. This ensures that the model is never trained on future transactions when predicting fraud for a given point in time.

2.1.3 Temporal Feature Engineering

Temporal features were engineered on the combined chronological stream of training and validation data. Because the features are computed using only past transactions relative to each row, validation rows are never based on information from the future, matching a real time deployment.

Features were constructed by defining entity groupings that capture different behavioral perspectives on each transaction:

- **card1:** Transaction history for the primary card identifier.

- **card1_addr1:** Interaction between card and billing address.
- **card1_card2:** Interaction between primary and secondary card identifiers.
- **email_p:** Transaction history grouped by purchaser email domain.

For each entity grouping, rolling statistics were computed over three time windows: 1 hour (3,600s), 1 day (86,400s), and 7 days (604,800s). These aggregations capture short-term changes, daily spending patterns, and weekly behavioral trends. This allowed the model to consider temporal context when distinguishing legitimate transactions from fraudulent ones.

2.1.4 Missing Value Handling

Extreme Missingness Removal

Columns with more than 90% missing values were dropped from the dataset.

Imputation

A custom `MissingValueImputer` was implemented to learn fill values from the training set and apply them to both the training and validation sets.

- **V-features with >50% missing:** Filled with a sentinel value of -999 to explicitly signal missingness to the model, preserving the informational content of the missing pattern itself.
- **V-features with $\leq$50% missing:** Imputed with the training set median.
- **Other numeric columns:** Imputed with the training set median, falling back to -999 if the median was undefined.
- **Boolean columns:** Imputed with the training set mode, defaulting to `False` if no mode was available.
- **Datetime columns:** Imputed with the training set median timestamp.
- **Categorical, string, and object columns:** Filled with the placeholder value `"Unknown"`.

2.1.5 Categorical Encoding

Categorical columns are encoded using a `CategoricalEncoder` fit on the training set. The encoder maps each category to its mean fraud rate observed in the training data. This method was selected due to the high cardinality of features such as email domains, which ruled out the possibility of one-hot encoding. The encoder is applied to the validation set using the mappings from the training set.

2.1.6 Feature Selection

A three-stage feature selection process reduces the feature space to the most informative predictors. First, near-zero-variance filtering is applied to remove features with low variability. Second, pairwise correlation filtering targets highly correlated feature groups and removes all but the feature with the highest association to the fraud label. Third, mutual information filtering retains the features with the highest nonlinear dependence on the target column. All three stages are run on the training set to determine a feature list, which is then applied identically to the validation set.

2.1.7 Final Preparation

After feature selection, the target column (`isFraud`) and `TransactionID` are separated from the feature matrices. All remaining feature columns are cast to 32-bit floating point to reduce memory usage during model training. The resulting training and validation splits are serialized to disk for reproducible downstream modeling.

2.2 Model Selection

Four gradient-boosted tree models were evaluated as candidates for the final fraud detection system: LightGBM, XGBoost, CatBoost, and Histogram-Based Gradient Boosting (HistGradientBoosting). Gradient-boosted decision trees were chosen as the model family because they are natively suited to tabular, mixed-type data with high cardinality categorical features, handle class imbalance effectively through `scale_pos_weight` tuning, and support SHAP-based explainability through fast, exact TreeExplainer calculations. Deep learning alternatives were considered but deprioritised, as the structured tabular nature of the IEEE-CIS dataset does not present the unstructured patterns that typically motivate deep learning, and the additional complexity would impede interpretability without clear performance gains.

HistGradientBoosting was excluded from the final ensemble because it employs the same histogram-based gradient boosting algorithm as LightGBM, meaning the two models learn similar decision boundaries from the same feature space. Including both would reduce ensemble diversity without adding complementary predictive signal. The three retained models differ structurally: LightGBM uses leaf-wise growth with gradient-based one-side sampling, XGBoost uses level-wise growth with explicit L1 and L2 regularization, and CatBoost uses symmetric trees with ordered boosting. This architectural diversity ensures each model captures distinct patterns in the data. Model performance is reported and discussed in Section 3.1.

2.3 Model Development and Training

Model training follows a consistent train–evaluate–save workflow across all model implementations. First, the preprocessed train/validation splits are loaded from disk. Each model is initialized with a baseline hyperparameter configuration (with optional overrides), and class imbalance handling is applied through class-weight scaling when supported. Models are then fit on the training set while monitoring validation performance to enable early stopping and avoid overfitting.

After fitting, each model produces probability scores on both the training and validation sets. These scores are thresholded into binary class predictions and used to compute evaluation metrics including ROC-AUC, precision, recall, and F1-score. Each trained model artifact is serialised to disk for use in the ensemble inference pipeline.

The final system combines the three models into a soft-voting weighted ensemble. Rather than taking a hard majority vote, the ensemble computes a weighted average of each model's predicted fraud probability: LightGBM contributes 35% of the final score, XGBoost contributes 40%, and CatBoost contributes 25%. Weights were assigned in proportion to each model's recall performance on the validation set, reflecting the project's priority of maximising fraud detection sensitivity. XGBoost receives the highest weight due to its superior recall, while LightGBM's higher precision serves as a counterbalance that reduces false positives in the combined score.

A fixed decision threshold of 0.2920 was selected by evaluating the precision-recall curve of the ensemble's combined probability scores. This threshold was chosen to maximise recall while maintaining precision at or above 0.25, a floor chosen to reflect an operationally realistic false positive rate for a fraud review system. The final ensemble achieves a ROC-AUC of 0.9119, recall of 0.7197, and precision of 0.2500 on the validation set, outperforming all individual models on the recall metric while improving precision relative to XGBoost alone.

SHAP values for the explainability dashboard were computed using the LightGBM component of the ensemble via SHAP's TreeExplainer. This approach provides exact, closed-form explanations with linear time complexity relative to the number of trees, making it feasible to compute per-transaction feature contributions at inference time. LightGBM was selected as the explanation model rather than XGBoost or CatBoost because its leaf-wise tree structure produces the most stable SHAP attributions on this dataset, and because it has the best precision among the three models, meaning its individual predictions are the most trustworthy to explain.

2.4 Dashboard

The dashboard serves a tri-purpose role: an interactive monitoring tool for fraud analysts to investigate flagged transactions, a consumer-facing interface that gives cardholders visibility into their fraud risk and spending patterns, and

a demonstration platform showing how the ensemble model predictions and SHAP-based explainability can be surfaced to end users. Built with Streamlit and Plotly, it translates raw model outputs into intuitive visualizations accessible to both technical and non-technical audiences. The dashboard reads from two pre-computed output files: predictions.csv, which contains fraud probability scores generated by the trained ensemble on the validation split, and shap_values.csv, which contains pre-computed SHAP explanations. Using the validation split rather than training data ensures honest model evaluation: predictions shown to users reflect performance on data the model has never seen during training. A persistent sidebar user and card selector allows analysts to switch between demo accounts (Alice, Bob, Carol, and Dave each with two associated cards) without losing context across pages.

2.4.1 Fraud Detection

The Fraud Detection pages provide both a high-level account summary and an operational investigation interface. The Home page displays aggregate statistics including total transaction volume, overall fraud rate, and a chronological transaction timeline, allowing users to identify unusual spikes in activity at a glance. The Fraud Alerts page presents a filterable, ranked table of transactions flagged as high-risk by the ensemble model. Each row displays the transaction amount, timestamp, merchant context, and the model's fraud probability score, enabling analysts to triage alerts efficiently. Transactions are surfaced when their predicted fraud probability exceeds a threshold calibrated on the validation set to balance precision and recall in accordance with the fraud detection objective.

2.4.2 Explanation

The Explanation page addresses a critical requirement in regulated financial environments: the ability to justify why a transaction was flagged as fraudulent. Using pre-computed SHAP (SHapley Additive exPlanations) values, this page renders per-transaction feature importance charts that decompose the model's fraud probability into individual feature contributions. Users can select any flagged transaction and immediately see which features most strongly drove the prediction. Analysis of the SHAP outputs identified the top three fraud drivers in the dataset as C2 (the count of addresses associated with a card), TransactionAmt, and card6 (indicating whether the card is credit or debit), findings consistent with known fraud patterns in the literature. Feature contributions are displayed in a horizontal bar format, making explanations interpretable without requiring familiarity with SHAP methodology. SHAP values are pre-computed rather than generated on demand, keeping page load times within the system's sub-50 millisecond latency target.

2.4.3 Spending Breakdown

The Spending Breakdown page shifts focus from fraud detection to financial insight, providing cardholders with a breakdown of their transaction history across time and spending categories. Interactive Plotly charts display transaction frequency distributions, average spend by time of day, and month-over-month spending trends. This page fulfills the consumer-facing dimension of the dashboard: giving users the context to understand not only whether fraud occurred but how their overall spending behavior compares to historical patterns. Anomalous spending periods are visually distinguished, bridging the fraud alerts and the underlying behavioral data that drives model predictions.

2.4.4 Risk Score

The Risk Score page presents a longitudinal view of each card's aggregated fraud risk over time. A composite risk score is calculated by mathematically aggregating the fraud probability scores of all transactions associated with a given card, providing a single interpretable metric that summarizes how risky a card's overall transaction history has been. The tracker displays this score as a time-series chart, enabling both cardholders and analysts to monitor whether a card's risk profile is improving or deteriorating and to correlate risk spikes with specific flagged transactions identified on the Fraud Alerts page.

3. Results and Discussion

3.1 Model Performance Comparison

Table 3.1 reports validation performance for each individual model and the final weighted ensemble.

Table 3.1: Validation set performance for individual models and the final weighted ensemble at threshold 0.2920.

Model	ROC-AUC	Recall	Precision	F1-Score
LightGBM	0.9103	0.4734	0.5271	0.4988
XGBoost	0.9052	0.7281	0.2246	0.3433
CatBoost	0.9103	0.6021	0.3509	0.4434
Ensemble (final)	**0.9119**	**0.7197**	**0.2500**	**0.3711**

Ensemble weights: XGBoost 40%, LightGBM 35%, CatBoost 25%. Threshold fixed at 0.2920.

The results reveal a pronounced precision-recall tradeoff across the three models. LightGBM achieves the highest precision (0.53) but lowest recall (0.47), reflecting conservative fraud flagging. XGBoost achieves the highest recall (0.73) but lowest precision (0.22), a consequence of its aggressive class imbalance correction. CatBoost occupies the middle ground with recall of 0.60 and precision of 0.35. All three models achieve similar ROC-AUC scores in the range of 0.905–0.910, confirming that the divergence in recall and precision arises from threshold and class-weight calibration rather than differences in learning capacity.

No single model simultaneously achieves acceptable recall and precision, which is the primary motivation for the weighted ensemble. The ensemble achieves ROC-AUC of 0.9119, recall of 0.7197, and precision of 0.2500, outperforming LightGBM on recall by 0.25 points and outperforming XGBoost on precision by 0.025 points. This confirms that combining models with complementary precision-recall profiles produces a more balanced operating point than

any individual model achieves alone.

3.2 Best Model Metrics

Table 3.2 presents the confusion matrix for the final ensemble at the chosen decision threshold of 0.2920 on the 118,108-transaction validation set.

Table 3.2: Confusion matrix for the final ensemble on the validation set at threshold 0.2920. The validation set contains 114,044 legitimate and 4,064 fraudulent transactions.

	Predicted Legitimate	**Predicted Fraud**
Actual Legitimate	105,269 (TN)	8,775 (FP)
Actual Fraud	1,139 (FN)	2,925 (TP)

The ensemble correctly clears 105,269 legitimate transactions (92.3% of all legitimate transactions) while flagging 8,775 as suspicious. Of the 4,064 fraudulent transactions, 2,925 are correctly identified and 1,139 are missed. The false negative rate of 28% represents the primary limitation of the current system and reflects the inherent difficulty of detecting rare fraud patterns in a highly imbalanced dataset where fraudulent transactions constitute only 3.44% of the validation set.

3. Bibliography

[1] M. Khalid *et al.*, "Enhancing credit card fraud detection: An ensemble machine learning approach," *Journal of Risk and Financial Management*, vol. 8, no. 1, p. 6, 2024.

[2] A. Compagnino *et al.*, "An introduction to machine learning methods for fraud detection," *Applied Sciences*, vol. 15, no. 21, p. 11787, 2025.

[3] A. Unknown, "An integrated multistage ensemble machine learning model for fraudulent transaction detection," *Journal of Big Data*, 2024.

[4] A. Unknown, "Machine learning for credit scoring and loan default prediction using behavioral and transactional financial data," 2025. ResearchGate Preprint.

[5] A. Unknown, "Application of natural language processing in financial risk detection," *arXiv preprint arXiv:2406.09765*, 2024.

[6] "Ieee-cis fraud detection dataset."